Exercise 9: *PyVSC* Coverage for *Saturation Filter* testbench

Objective: *Introduction to the coverage collection in a UVM testbench using the PyVSC library.*

Task: *Implementation of the coverage collector class for the Saturation Filter testbench.*

The files for this exercise are in `<ROOT>/lecture06/exercises/E09_sat_coverage/`. This directory contains the `Makefile` and the testbench files. In the following sections, file names are relative to `tb/`.

Updates to *sSDT UVC*

The *sSDT UVC* provided for this exercise has been extended with the `uvc_ssdt_monitor` and `uvc_ssdt_coverage` components. The producer and consumer agents will now sample the `data` values of their respective interfaces into a coverpoint.
See `<ROOT>/lecture06/exercises/exercise_general_info.pdf` for an overview of the updated *sSDT UVC*.

Part 1: Collecting Some coverage

`sat_tb_base_test.py` contains a new class method: `create_pyvsc_coverage_report`. This method writes the collected coverage of a test to a text report and an XML coverage database. It is called in the `report_phase` of the base test. Since every test extends `sat_tb_base_test`, the report phase creates both of the following files for each test in `sim_build/`:

- `<test-name>_cov.txt`: a human-readable coverage report.
- `<test-name>_cov.xml`: a coverage database for the `PyUCIS-viewer` tool.

Running and Inspecting the Tests

`tests/` and `vseqs/` contain the test and virtual sequence solutions from exercise 08, you can alternatively replace them with your own tests from exercise 08.

Before running the tests inspect the implementation of the covergroup in `uvc/ssdt/src/uvc_ssdt_coverage.py`. Which bins are defined for `cp_data`, and which `data` values does each bin cover?

Run the tests and inspect the generated coverage. You can either:

- Open the `.txt` files in a text editor.
- Or open an individual `.xml` coverage database with `PyUCIS-viewer`. For example, to inspect the default test, run:

```
(.venv) [<username>@<servername> tb]$ pyucis-viewer
↪  sim_build/sat_tb_default_test_cov.xml
```

Compare the coverage collected by the producer and consumer agents. Inspect which `data` bins each test covers.

Merging Coverage Reports

The individual reports show the coverage achieved by one test. You can also merge the coverage of multiple tests. To merge the coverage of all coverage reports in `sim_build` run:

```
(.venv) [<username>@<servername> tb]$ make coverage-merge
```

The `coverage-merge` target merges the individual coverage databases from `sim_build/` into `sim_build/merge_cov.xml`.

To merge the coverage databases and immediately open the merged result in `PyUCIS-viewer`, run:

```
(.venv) [<username>@<servername> tb]$ make coverage-view-merge
```

Part 2: Adding *Saturation Filter* coverage

The *sSDT* coverage collectors sample the `data` fields of the input and output interfaces, but they do not cover the *Saturation Filter*-specific `ovf` signal. In this part, you will add a separate covergroup for the *Saturation Filter*. It will cover the value of `ovf` and the number of transactions observed since the previous overflow.

Saturation Filter Covergroup

Before implementing the covergorup, read the *pyvsc* at `https://pyvsc.readthedocs.io/en/latest/index.html` and answer the following questions:

- How does `with_sample` define the arguments and argument types of a covergroup's `sample` method?
- How is a coverpoint created for one of these sample arguments?
- How are named bins defined for a coverpoint, and what is the difference between `vsc.bin` and `vsc.bin_array`?

In `sat_coverage.py`:

- Implement a covergroup named `covergroup_sat`:
- Decorate it with `@vsc.covergroup`.
- The covergroup must accept the following values through its sample method:
  - `ovf` as a one-bit value.
  - `count` as a 32-bit unsigned value.
- Define two coverpoints in the covergroup:
  - `cp_ovf` with one bin for `ovf == 0` and one bin for `ovf == 1`.
  - `cp_count` with one bin for each transaction count from 1 through 100. Use `vsc.bin_array` to create the bins.

Integrating the Covergroup

The provided `monitor_loop_ovf` coroutine in `sat_tb_base_test.py` samples the `ovf` signal and counts the transaction until an overflow occurs.

In `sat_tb_base_test.py`:

- Import the covergroup from `sat_coverage.py`.
- Add a `sat_cov` handle to the class constructor.
- In `build_phase`, instantiate the covergroup.
- In `monitor_loop_ovf` pass the value of the overflow signal from the DUT and the transaction counter to the sample method of your covergroup.

Running and Inspecting the Tests

Run the tests again and inspect the coverage reports. Which tests cover both values of `ovf`? Which transaction-count bins are covered by each test, and how does this relate to the generated stimuli?

Part 3: Crosses

The individual coverpoints show which overflow values and transaction counts were observed. A cross additionally measures which combinations of these values occurred together.

In `sat_coverage.py` add a cross of the overflow and transaction count coverpoints.

Run the tests again and inspect the cross coverage in both the individual and merged reports. How many cross bins are created from the two `ovf` bins and the 100 transaction-count bins? Which combinations remain uncovered?